

# JSKim ユーザーガイド

v0.6.0

一般公開ドキュメント

## このガイドについて

---

主な読者は次のとおりです。

HTML、CSS、JavaScript の基本的な知識はあるが、  
JSKim および Nunjucks を初めて利用するユーザー

このガイドでは、次のことを学べます。

- JSKim が何をするツールか
- project の作り方と build / 開発サーバーの動かし方
- 公式 sample の構成と Dashboard / CRUD / Wizard の読み方
- CLI、設定、files pipeline、Nunjucks の使い方
- 開発機能と代表的なエラーの対処
- JSKim の適用範囲と制限

説明は公式の静的 UI sample (`src/sample/`) とあわせて読む前提です。  
creator (`create-jskim`) で生成した project にも、同じ sample が含まれます。

## 読み方

---

### 初めて使う場合

次の順がわかりやすいです。

1. JSKimとは
2. はじめ方
3. プロジェクト構成
4. 基本的な開発workflow
5. 公式 sample を見ながら Dashboard例 以降

必要に応じて CLI / 設定 / files pipeline / Nunjucks の章を参照してください。

# 目次

---

- [01. JSKimとは](#)
- [02. はじめ方](#)
- [03. プロジェクト構成](#)
- [04. 基本的な開発workflow](#)
- [05. CLIリファレンス](#)
- [06. 設定](#)
- [07. files pipeline](#)
- [08. Nunjucksの使い方](#)
- [09. 開発機能](#)
- [10. エラーとトラブルシューティング](#)
- [11. Dashboard例](#)
- [12. CRUD例](#)
- [13. Wizard例](#)
- [14. 制限事項](#)

# JSKimとは

---

この章では、JSKim がどのようなツールか、何を解決し、どのような用途に向くかを説明します。

## JSKim の概要

JSKim は、Nunjucks を使って静的 HTML を生成する開発環境です。

source (`sourceDir`) を管理し、build によって output (`outputDir`) を生成します。layout や component を再利用でき、CSS や画像などの asset もまとめて扱えます。開発中は `watch` / `serve` / `dev` を使い、配布結果は通常の静的ファイルです。

「framework」と呼ぶこともできますが、backend framework や SPA framework ではありません。テンプレートと静的ファイルを整理し、build 結果を作るためのツールです。

## 解決しようとする問題

静的な HTML サイトを手作業だけで増やすと、次のような問題が起きやすくなります。

- 複数ページで header / footer などの共通部分が重複する
- ページ数が増えると、共通箇所の修正漏れが起きやすい
- 編集用の source と、配布用の output が混ざりやすい
- 静的 asset のパス管理が煩雑になる
- build / watch / 開発サーバーの手順が統一されない
- template の誤りに気づくのが遅れる

JSKim は、これらの作業を `jskim.config.js` を基準にした一連のコマンドとしてまとめます。

## 基本的な処理の流れ

推奨設定である files pipeline では、大まかに次の流れで処理します。

```
src/sample/
|
├ pages/ 配下の *.njk — render (末尾 .njk を外す) ┐
├ pages/ 配下の CSS / 画像 — copy —————┘
└ layouts / components — template root (直接出力しない) |
                                                                ▼
                                                                dist/sample/
```

`layouts` や `components` は `extends` / `include` 用の template root です。それ自体は output されません。詳細な規則は [files.pipeline](#) を参照してください。

## JSKim に向いている project

次のような project に向いています。

- 複数の静的 HTML ページを持つ website
- 共通 layout / component を使う project
- build 結果を静的 hosting や既存サーバーへ配置する project
- frontend bundler を使わず、HTML / CSS / JS asset を整理したい project

## JSKim の範囲外

次の用途は JSKim の範囲外です。

- backend application framework ではない
- database 機能はない
- API server 機能はない
- React / Vue のような SPA framework ではない
- JavaScript bundler ではない
- 既存 HTML の自動取り込みや移行ツールではない
- 実際の CRUD / Wizard 業務ロジックは提供しない

公式 sample の CRUD / Wizard は、画面構成を示す静的 UI sample です。保存や送信などの application 処理はありません。

## JSKim と Nunjucks の関係

役割を分けると次のとおりです。

- **Nunjucks** — template の記法と render engine
- **JSKim** — project / 設定 / files / build / watch / serve / dev をまとめる実行環境

Nunjucks の文法すべてが JSKim 独自機能というわけではありません。

このユーザーガイドでは、JSKim の利用で実務的に必要な文法を中心に扱います。

## 公式 sample の紹介

公式 sample は次の画面グループで構成されています。

| 画面グループ    | 役割                            |
|-----------|-------------------------------|
| Portal    | sample 全体の入口。各画面グループへの導線をまとめる |
| Dashboard | 共通 layout と静的データを使った管理画面の見た目例 |
| CRUD      | 一覧・詳細・登録・編集・削除確認などの画面構成例      |

| 画面グループ | 役割                    |
|--------|-----------------------|
| Wizard | 入力・確認・完了の 3 ステップ画面構成例 |

いずれも静的 UI です。API 通信や入力の永続化はありません。

代表的な source:

- [Portal](#)
- [Dashboard](#)
- [CRUD](#)
- [Wizard \(入力\)](#)

読み方は [Dashboard例](#)、[CRUD例](#)、[Wizard例](#) を参照してください。

次の章では、project を作成して sample をブラウザで確認する手順を説明します。

# はじめ方

この章では、JSKim の project を作成し、ブラウザで公式 sample を確認し、production build まで実行する手順を説明します。

## 事前準備

JSKim engine ( `@ywal123456/jskim` ) と creator ( `create-jskim` ) は、いずれも Node.js **18 以上** を想定しています ( `package.json` の `engines.node` が `>=18` )。

package manager は npm / pnpm / yarn を公式に案内しています。OS として Windows / macOS / Linux を特別に制限してはいません。パス処理は Node の `path` を使います。

あらかじめ Node.js と、利用する package manager がコマンドとして認識できる状態にしてください。

## Project の作成

推奨の作成コマンドは次のとおりです。

```
npm create jskim@latest my-project
```

他の package manager の例:

| package manager | 作成コマンド                                          |
|-----------------|-------------------------------------------------|
| npm             | <code>npm create jskim@latest my-project</code> |
| pnpm            | <code>pnpm create jskim my-project</code>       |
| yarn            | <code>yarn create jskim my-project</code>       |

代替として、次も利用できます。

```
npx create-jskim my-project
```

ディレクトリ名を省略すると、プロジェクト名を尋ねます。空のまま Enter すると既定値 `jskim-project` を使います。

空ではない既存ディレクトリは上書きしません。

## 生成結果

生成直後の主要な構成は次のとおりです。

```
my-project/  
└─ package.json
```

```
├─ jskim.config.js
├─ README.md
├─ .gitignore
├─ src/
│   └─ sample/
│       ├── layouts/
│       ├── components/
│       └─ pages/
```

`package.json` は creator が生成します。engine dependency は `@ywal123456/jskim (^0.6.0)` です。  
`src/sample/` には公式の静的 UI sample (Portal / Dashboard / CRUD / Wizard) が含まれます。

## Dependency のインストール

creator は **自動で install を実行しません**。また `git init` も実行しません。

生成完了時の案内は、実行した package manager に合わせて install / 開発コマンドを表示します。検知できない場合は npm 向けの案内になることがあります。

project ディレクトリへ移動してから、次を実行してください。

```
npm install
```

```
pnpm install
```

```
yarn install
```

## 開発サーバーの実行

生成された `package.json` の `dev` script は `jskim dev sample` です。

```
npm run dev
```

package manager ごとの例:

| package manager | 開発コマンド                   |
|-----------------|--------------------------|
| npm             | <code>npm run dev</code> |
| pnpm            | <code>pnpm dev</code>    |
| yarn            | <code>yarn dev</code>    |

既定の開発サーバー URL は次です。

```
http://127.0.0.1:3000/
```

ブラウザでは Portal が表示されます。sidebar やカードから Dashboard / CRUD / Wizard へ移動できます。

source を保存すると再 build され、ブラウザへ反映されます。template などに誤りがあると、ブラウザ上の overlay に表示されることがあります。

詳細は [開発機能](#) を参照してください。

## 最初の修正

まずは Portal の説明文を変更してみます。

編集する file:

- [src/sample/pages/index.html.njk](#)

次の lead 文を探します。

```
<p class="lead">
  JSKim の files pipeline で画面構成、template 分割、asset 配置を確認できる
  静的 UI sample です。
</p>
```

文言を少し変えて保存し、ブラウザを確認してください。Portal の説明が更新されていれば、source から output への流れを確認できています。

## Production build

配布用の output を作るには次を実行します。

```
npm run build
```

生成 script は `jskim build sample` です。結果は次のディレクトリに出力されます。

```
dist/sample/
```

ここには通常の HTML / CSS / SVG などの静的ファイルが並びます。

どこへ配置するかは hosting 環境によって異なるため、特定の提供元の手順はこのガイドでは扱いません。

## 主なコマンドのまとめ

| 目的                    | 例 (npm)                    |
|-----------------------|----------------------------|
| dependency のインストール    | <code>npm install</code>   |
| 開発サーバー                | <code>npm run dev</code>   |
| production output の生成 | <code>npm run build</code> |



`watch`（監視して再 build）と `serve`（output の静的配信のみ）も生成 script に含まれます。  
使い分けは [基本的な開発workflow](#)、構文の詳細は [CLIリファレンス](#) を参照してください。

## うまく動かないとき

よくある原因は次のとおりです。

- 既定 port（`3000`）が他のプロセスで使われている
- `install` を実行していない
- project ルート以外のディレクトリでコマンドを実行している
- Node.js / package manager が PATH 上で認識されていない

エラーメッセージごとの対処は [エラーとトラブルシューティング](#) を参照してください。

# プロジェクト構成

---

この章では、`create-jskim` が生成する project の directory 構成と、各要素の役割を説明します。

## 全体構成

生成直後のルート構成は次のとおりです。

```
my-project/
├─ jskim.config.js
├─ package.json
├─ README.md
├─ .gitignore
├─ src/
│  └─ sample/
└─ dist/      ← build 後に生成されることがある
    └─ sample/
```

`dist/` は build を実行するまで存在しない場合があります。source は `src/sample/`、output は `dist/sample/` です。

`src/sample/` の概要は次のとおりです。

```
src/sample/
├─ layouts/
│  └─ base.njk
├─ components/
│  ├─ header.njk
│  ├─ sidebar.njk
│  ├─ breadcrumb.njk
│  ├─ footer.njk
│  └─ wizard-steps.njk
└─ pages/
   ├─ index.html.njk
   ├─ assets/
   │  ├─ css/common.css
   │  └─ img/logo.svg
   ├─ dashboard/
   ├─ crud/
   └─ wizard/
```

## jskim.config.js

`jskim.config.js` は project の定義です。動作の基準はこのファイルにあります。

公式 sample では、おおむね次の考え方で設定されています。

- `sourceDir` — source のルート (例: `src/sample`)
- `outputDir` — output のルート (例: `dist/sample`)

- `files` — `pages` 配下をまとめて処理する pipeline (例: `{ from: 'pages', to: '' }`)
- `templates` — `extends` / `include` 用の検索パス (例: `layouts`, `components`)
- `data` — 全 template へ渡す共通データ (例: `site.name`)

設定の一部 (概念) :

```
defaults: {
  files: [{ from: 'pages', to: '' }],
  templates: ['layouts', 'components'],
  data: {
    site: {
      name: 'JSKim UI Sample',
      language: 'ja',
      themeColor: '#222222',
    },
  },
}
```

オプションの詳細は [設定](#) を参照してください。

## src/sample

`src/sample/` は公式 sample の source root です。

この repository の sample と、creator が生成する sample は同じ構成です。

主な役割:

- `layouts/` — 文書全体の共通枠
- `components/` — header など再利用部品
- `pages/` — 実際に output されるページと asset

## layouts

共通 layout は次の file です。

- [base.njk](#)

HTML document の骨格 (`title`、共通 CSS、header、sidebar、breadcrumb、footer) を定義します。

各ページは `{% extends "layouts/base.njk" %}` でこの layout を継承し、`{% block content %}` などに本文を書きます。

`layouts` は template root のため、それ自体は output されません。

## components

components は次のとおりです。

| file                             | 役割            |
|----------------------------------|---------------|
| <a href="#">header.njk</a>       | 上部ヘッダーとブランド表示 |
| <a href="#">sidebar.njk</a>      | 画面グループへのナビ    |
| <a href="#">breadcrumb.njk</a>   | パンくずリスト       |
| <a href="#">footer.njk</a>       | フッター          |
| <a href="#">wizard-steps.njk</a> | Wizard の手順表示  |

layout から `{% include "components/header.njk" %}` のように読み込みます。  
components も template root のため、独立した HTML ページとしては output されません。

## pages

`pages/` 配下が実際の output 対象です。

- `*.njk` は末尾の `.njk` だけを外して render されます (例: `index.html.njk` → `index.html`)
- CSS / SVG などそれ以外は copy されます
- `pages/` からの相対構造は output でも維持されます

公式 sample のページ構成:

| グループ       | 内容                                |
|------------|-----------------------------------|
| Portal     | <code>pages/index.html.njk</code> |
| dashboard/ | Dashboard 画面と page-local CSS      |
| crud/      | 一覧・詳細・登録・編集・削除・完了などの画面群           |
| wizard/    | 入力・確認・完了の画面群                      |

## 共通 asset

共通で使う asset の例:

| source                                      | output                                         |
|---------------------------------------------|------------------------------------------------|
| <a href="#">pages/assets/css/common.css</a> | <code>dist/sample/assets/css/common.css</code> |
| <a href="#">pages/assets/img/logo.svg</a>   | <code>dist/sample/assets/img/logo.svg</code>   |

深い階層のページからも同じ共通 asset を参照するため、layout では `rootPath` を使います。

```
<link rel="stylesheet" href="{{ rootPath }}assets/css/common.css">
```

`rootPath` は output 上のページ位置に応じて、output root までの相対パスを表します。  
詳細は [files pipeline](#) を参照してください。

## Page-local asset

画面グループ専用の CSS 例:

| source                                      | output                                         |
|---------------------------------------------|------------------------------------------------|
| <a href="#">dashboard/.../dashboard.css</a> | dist/sample/dashboard/assets/css/dashboard.css |
| <a href="#">crud/.../crud.css</a>           | dist/sample/crud/assets/css/crud.css           |
| <a href="#">wizard/.../wizard.css</a>       | dist/sample/wizard/assets/css/wizard.css       |

page-local CSS は、その画面グループからの相対パスで参照します。

```
<link rel="stylesheet" href="assets/css/dashboard.css">
```

### Warning

次のように `rootPath` を付けて page-local CSS を参照しないでください。

```
{{ rootPath }}assets/css/dashboard.css
```

`rootPath` は output root (`dist/sample/`) 基準です。  
Dashboard 用 CSS は `dist/sample/dashboard/assets/...` にあるため、  
root 基準のパスでは見つかりません。

共通 asset と page-local asset の比較:

| 種類               | 置き場所の例                                     | 参照の考え方                                       |
|------------------|--------------------------------------------|----------------------------------------------|
| 共通 asset         | <code>pages/assets/...</code>              | <code>{{ rootPath }}assets/...</code>        |
| page-local asset | <code>pages/dashboard/assets/...</code> など | 同一グループからの相対パス<br>( <code>assets/...</code> ) |

## dist/sample

`dist/sample/` は build の output です。

- 手で編集する source ではありません
- 次の build で上書き・再生成されることがあります
- 配布対象として使える通常の静的ファイルです
- `layouts` / `components` はここへ出力されません

## Source と output の対応

| source                                    | 処理            | output                             |
|-------------------------------------------|---------------|------------------------------------|
| pages/index.html.njk                      | render        | index.html                         |
| pages/assets/css/common.css               | copy          | assets/css/common.css              |
| layouts/base.njk                          | template root | 直接 output なし                       |
| components/header.njk                     | template root | 直接 output なし                       |
| pages/crud/detail.html.njk                | render        | crud/detail.html                   |
| pages/dashboard/assets/css/dashboards.css | copy          | dashboard/assets/css/dashboard.css |

## どこを編集するか

目的ごとの目安です。

| 目的            | 主な編集先                                       |
|---------------|---------------------------------------------|
| ページ本文         | pages/**/*.*.html.njk                       |
| 共通 layout     | <a href="#">layouts/base.njk</a>            |
| 共通 component  | components/*.*.njk                          |
| 全体の見た目        | <a href="#">pages/assets/css/common.css</a> |
| 画面グループの見た目    | 各グループの <code>assets/css/*.*.css</code>      |
| build やサーバー設定 | jskim.config.js                             |

日常の開発の流れは [基本的な開発workflow](#)、  
CLI と設定の詳細は [CLIリファレンス](#) / [設定](#) を参照してください。

# 基本的な開発workflow

この章では、`build` / `watch` / `serve` / `dev` の違いと、日常開発での選び方を説明します。  
個別 option の詳細は [CLIリファレンス](#) を参照してください。

## 全体の流れ

開発の基本的な流れは次のとおりです。

```
src/sample/ を編集
↓
build または watch
↓
dist/sample/ へ output
↓
serve または dev で確認
```

`dev` は、初回の `build`、変更監視 (`watch`)、静的サーバー (`serve`)、開発用の `live reload` をまとめて行う統合コマンドです。

日常の画面確認では、まず `dev` を使うのが簡単です。

## build

`build` は、いまの source から output を **1回** 生成して終了します。

用途の例:

- production 向け output の確認
- 配布前の最終生成
- CI など、一度だけ成果物が欲しい場面

生成 project では次を使います。

```
npm run build
```

これは `jskim build sample` と同じです。

成功すると process は終了します (exit code `0`)。

設定や render の失敗などがあると、エラーを出して終了します (exit code `1`)。

既定では `build.clean` が `true` のため、build 前に `outputDir` を削除してから再生成します。

詳細は [設定](#) と [files.pipeline](#) を参照してください。

## watch

`watch` は source の変更を監視し、変更のたびに output を更新します。

```
npm run watch
```

これは `jskim watch sample` と同じです。

特徴:

- 初回に build を行い、その後も監視を続ける
- 開発用の静的サーバーは起動しない
- ブラウザへの配信や live reload は行わない
- `jskim.config.js` の変更も検知できる（概要。詳細は [開発機能](#)）

「output だけを更新し続けたい」「サーバーは別手段で見る」場合に向きます。

## serve

`serve` は、すでに生成された `outputDir` を静的サーバーとして公開します。

```
npm run serve
```

これは `jskim serve sample` と同じです。

特徴:

- build や watch は自動では行わない
- source を編集しても自動では再 build しない
- 既定の host / port は `127.0.0.1` / `3000`
- CLI で `--host` / `--port` を指定できる
- `--open` は サポートしていない

既存の `dist/` を手元で確認したいときに使います。

最新の source を反映したい場合は、先に `build` するか、`dev` を使ってください。

## dev

`dev` は開発用の統合コマンドです。

```
npm run dev
```

これは `jskim dev sample` と同じです。

実際には次をまとめて行います。



1. 初回 build
2. source / config の監視と再 build
3. `outputDir` の静的サーバー
4. 開発用 live reload (既定で有効)

既定の URL:

```
http://127.0.0.1:3000/
```

使える option の例:

```
jskim dev sample --host 127.0.0.1 --port 4000 --open
```

- `--host` / `--port` — 待ち受け設定 (config より優先)
- `--open` — サーバー起動後にブラウザを開く (`dev` のみ)

source や設定の誤りがあっても、開発中の process をすぐ終了させない設計です。  
error overlay や CSS soft reload の詳細は [開発機能](#) を参照してください。

## コマンドの選び方

| 目的                                | 推奨コマンド             |
|-----------------------------------|--------------------|
| 一度だけ output を作る                   | <code>build</code> |
| 変更を監視して output を更新し続ける            | <code>watch</code> |
| 既存の <code>dist/</code> をブラウザで確認する | <code>serve</code> |
| 編集しながらブラウザで開発する                   | <code>dev</code>   |

## 一般的な開発の順序

1. dependency を入れる

```
npm install
```

2. 開発サーバーを起動する

```
npm run dev
```

3. `src/sample/` を編集し、ブラウザで確認する
4. 配布用に最終 build する

```
npm run build
```

## 5. `dist/sample/`を確認する

配置先の手順は hosting 環境ごとに異なるため、このガイドでは扱いません。

## process の終了

`watch` / `serve` / `dev` は foreground で動き続けます。

終了するときは端末で `Ctrl+C` を送ります。

終了時は、サーバーや watcher を止めてから process が終わります。

成功時の停止では exit code `0` になります。

JSKim 自体に、外部 process manager のような常駐管理機能はありません。

## project が複数ある場合

`jskim.config.js` の `projects` に複数の名前がある場合は、コマンドに project 名を付けます。

```
jskim build siteA  
jskim watch siteB
```

project が **1 件だけ** のときは、名前を省略できます。

**2 件以上** あるときに省略するとエラーになります。

例外として、`build` だけは次で全 project を実行できます。

```
jskim build --all
```

`watch` / `serve` / `dev` に `--all` はありません。

詳細な構文は [CLIリファレンス](#) を参照してください。

# CLIリファレンス

この章では、現在の JSKim CLI (binary 名 `jskim`) がサポートするコマンドと option をまとめます。  
日常の選び方は [基本的な開発workflow](#) を参照してください。

## 基本構文

```
jskim <command> [project] [options]
```

`project` と option の順序は入れ替えられます。

```
jskim serve --port 4000 sample
jskim serve sample --port 4000
jskim dev sample --open --host 127.0.0.1
```

ヘルプと version:

```
jskim --help
jskim -h
jskim help
jskim --version
jskim -v
```

## コマンド一覧

| command            | 概要                                          |
|--------------------|---------------------------------------------|
| <code>build</code> | source から output を生成して終了する                  |
| <code>watch</code> | 初回 build のあと、変更を監視して再 build する              |
| <code>serve</code> | 既存の <code>outputDir</code> を静的サーバーで公開する     |
| <code>dev</code>   | build + watch + serve + live reload の開発サーバー |

## project 引数

`project` は `jskim.config.js` の `projects` にあるキー名です。

例:

```
projects: {
  sample: {
    sourceDir: 'src/sample',
    outputDir: 'dist/sample',
  },
}
```

```
  },  
}
```

このとき `jskim build sample` は `projects.sample` を対象にします。

| 状況                 | 動作           |
|--------------------|--------------|
| project が 1 件      | 名前を省略できる     |
| project が 0 件      | エラー          |
| project が 2 件以上で省略 | エラー（候補一覧を表示） |
| 存在しない名前を指定         | エラー          |

エラーメッセージの詳細な読み方は [エラーとトラブルシュート](#) を参照してください。

## build

```
jskim build sample  
jskim build --all
```

| 項目                               | 内容                                                 |
|----------------------------------|----------------------------------------------------|
| project                          | 1 件指定、または 1 件だけのとき省略可                              |
| <code>--all</code>               | <code>projects</code> の全キーを定義順に実行                  |
| project と <code>--all</code> の併用 | 不可                                                 |
| 実行順（ <code>--all</code> ）        | <code>Object.keys(projects)</code> の順              |
| 途中失敗                             | resolve / build 失敗があっても次の project へ続行              |
| 最終 exit code                     | 1 件でも失敗があれば <code>1</code> 、すべて成功なら <code>0</code> |
| 同一 / 入れ子の <code>outputDir</code> | <code>--all</code> では build 開始前に検査し、衝突なら中断         |

`--all` の衝突検査は、同じ path（Windows では大文字小文字を区別しない）や、厳密な祖先・子孫関係を拒否します。

似た prefix（例: `dist/site` と `dist/site-admin`）は許可されます。

### Tip

project 名が `all` のときは `jskim build all` が 1 件 build です。  
全件実行は必ず `jskim build --all` を使います。

## watch

```
jskim watch sample
```

| 項目                 | 内容                         |
|--------------------|----------------------------|
| project            | <code>build</code> と同じ選択規則 |
| 追加 option          | なし                         |
| <code>--all</code> | 非対応                        |
| 動作                 | 初回 build 後、監視を継続           |
| 終了                 | <code>Ctrl+C</code> など     |

静的サーバーは起動しません。

## serve

```
jskim serve sample  
jskim serve sample --host 127.0.0.1 --port 4000
```

| 項目                  | 内容                                                                                                             |
|---------------------|----------------------------------------------------------------------------------------------------------------|
| project             | <code>build</code> と同じ選択規則                                                                                     |
| <code>--host</code> | 待ち受け host                                                                                                      |
| <code>--port</code> | 待ち受け port                                                                                                      |
| 既定値                 | config の <code>serve.host</code> / <code>serve.port</code> (未設定時は <code>127.0.0.1</code> / <code>3000</code> ) |
| CLI と config        | CLI 指定が優先                                                                                                      |
| <code>--open</code> | 非対応                                                                                                            |
| <code>--all</code>  | 非対応                                                                                                            |
| build               | 自動では行わない                                                                                                       |

`port` は整数 `1` ～ `65535` である必要があります。

`host` は空でない文字列である必要があります。

## dev

```
jskim dev sample  
jskim dev sample --host 127.0.0.1 --port 4000 --open
```

| 項目                                        | 内容                            |
|-------------------------------------------|-------------------------------|
| project                                   | <code>build</code> と同じ選択規則    |
| <code>--host</code> / <code>--port</code> | <code>serve</code> と同じ        |
| <code>--open</code>                       | 起動後にブラウザを開く (listen 成功後に 1 回) |
| <code>--all</code>                        | 非対応                           |
| CLI と config                              | host / port は CLI が優先         |

`--open` でブラウザ起動に失敗した場合は warning を出し、開発サーバー自体は継続します。  
listen に失敗した場合はブラウザ起動を試みず、process は失敗終了します。

## option 要約

| option              | 使える command                           | 値               | 説明                  |
|---------------------|---------------------------------------|-----------------|---------------------|
| <code>--all</code>  | <code>build</code>                    | なし              | 全 project を順に build |
| <code>--host</code> | <code>serve</code> , <code>dev</code> | string          | 待ち受け host           |
| <code>--port</code> | <code>serve</code> , <code>dev</code> | string (検証後に整数) | 待ち受け port           |
| <code>--open</code> | <code>dev</code>                      | なし              | 起動後にブラウザを開く         |

短縮形の option alias (例: `-p`) はありません。

`--help` / `-h`、`--version` / `-v` はコマンド前のグローバル用途です。

## 引数解析の規則

次の場合はエラーになります。

- 不明な option
- その command で使えない option (例: `serve` に対する `--open`)
- option 値の欠落 (例: `--port` の直後が無い)
- option の重複
- `--port=4000` のような `=` 付き記法
- `--` 単独トークン
- positional 引数が 2 件以上
- `build` で project 名と `--all` の同時指定

option は project 名の前後どちらに置いても同じ結果です。

不正な port 値 (`0`、`65536`、小数、非数値など) は拒否されます。

## exit code

| 状況                                          | exit code                |
|---------------------------------------------|--------------------------|
| help / version                              | 0                        |
| 引数解析エラー                                     | 1                        |
| build 失敗（単一 / <code>--all</code> で 1 件以上失敗） | 1                        |
| watch / serve / dev の起動失敗                   | 1                        |
| <code>Ctrl+C</code> などでの正常停止                | 0                        |
| <code>dev --open</code> のブラウザ起動失敗           | process 自体は継続（失敗終了にはしない） |

長時間稼働中の `watch` / `dev` では、個別の再 build 失敗があっても process を維持します。  
詳細なエラー表示は [エラーとトラブルシューティング](#) と [開発機能](#) を参照してください。

## package script から使う

`create-jskim` が生成する `package.json` は、次の scripts を持ちます。

```
{
  "scripts": {
    "build": "jskim build sample",
    "watch": "jskim watch sample",
    "serve": "jskim serve sample",
    "dev": "jskim dev sample"
  }
}
```

そのため、日常作業では次で足ります。

```
npm run build
npm run watch
npm run serve
npm run dev
```

CLI option を足すときは、package manager の引数渡し規則に従ってください。

例（npm）：

```
npm run serve -- --port 4000
npm run dev -- --open --port 4000
```

# 設定

この章では、`jskim.config.js` で設定できる項目を説明します。  
既存の詳細 reference として [docs/configuration.md](#) もありますが、ここでは初学者向けの順で整理します。

## 設定ファイル

ファイル名はワークスペースルートの `jskim.config.js` です。  
コマンドを実行したディレクトリ（`process.cwd()`）をルートとして読みます。  
形式は CommonJS です。

```
module.exports = {
  defaults: {
    // 全 project 共通の既定値
  },
  projects: {
    // project 名ごとの設定
  },
};
```

ESM（`export default`）はサポートしていません。

## 最小の設定例

必須なのは、少なくとも 1 件の project と、その `sourceDir` / `outputDir` です。  
加えて、files mode では `files`、legacy mode では `render` が必要です。

推奨の最小例（files mode）：

```
module.exports = {
  defaults: {
    files: [{ from: 'pages', to: '' }],
    templates: ['layouts', 'components'],
  },
  projects: {
    sample: {
      sourceDir: 'src/sample',
      outputDir: 'dist/sample',
    },
  },
};
```

| key      | 必須 | 備考              |
|----------|----|-----------------|
| projects | 必須 | オブジェクト。1 件以上を推奨 |
| defaults | 任意 | 省略時は空オブジェクト扱い   |



| key                   | 必須            | 備考               |
|-----------------------|---------------|------------------|
| sourceDir / outputDir | project ごとに必須 | defaults 側に置いても可 |
| files または render      | mode に応じて必須   | 同時設定は不可          |

## 公式 sample の設定

この repository の設定は [jskim.config.js](#) です。

理解に必要な部分だけ抜粋します（filter / global の実装全体は省略しています）。

```
module.exports = {
  defaults: {
    files: [{ from: 'pages', to: '' }],
    templates: ['layouts', 'components'],
    data: {
      site: {
        name: 'JSKim UI Sample',
        language: 'ja',
        themeColor: '#222222',
      },
      samplePrice: 12000,
    },
    // nunjucks.filters / nunjucks.globals も定義できる
    build: { clean: true },
    watch: { debounce: 150 },
    serve: { host: '127.0.0.1', port: 3000 },
    dev: { liveReload: true },
  },
  projects: {
    sample: {
      sourceDir: 'src/sample',
      outputDir: 'dist/sample',
    },
  },
};
```

## トップレベルの構造

読み込み時に必須検査されるのは次です。

| key      | 型      | 必須 | 説明                |
|----------|--------|----|-------------------|
| projects | object | 必須 | project 名 → 設定    |
| defaults | object | 任意 | 全 project への共通既定値 |

それ以外のトップレベル key は、現在の loader では必須ではありません。

実際の build 設定は **defaults** と各 **projects.\*** をマージして使います。

## projects

`projects` は名前付き project の一覧です。

```
projects: {
  sample: { sourceDir: 'src/sample', outputDir: 'dist/sample' },
  docs: { sourceDir: 'src/docs', outputDir: 'dist/docs' },
}
```

- 各 project は独立した `sourceDir` / `outputDir` を持てます
- CLI の project 引数はこのキー名を指します
- 空の `projects` だとコマンド実行時にエラーになります

名前の文字種に特別な禁止規則はありませんが、CLI 引数として扱いやすい英数字名を推奨します。

## sourceDir

| 項目  | 内容                                       |
|-----|------------------------------------------|
| 型   | string                                   |
| 必須  | はい（マージ後）                                 |
| 既定値 | なし                                       |
| 基準  | ワークスペースルート（ <code>process.cwd()</code> ） |
| 検証  | 空不可。実在するディレクトリであること                      |
| 役割  | source のルート                              |

## outputDir

| 項目  | 内容                                                             |
|-----|----------------------------------------------------------------|
| 型   | string                                                         |
| 必須  | はい（マージ後）                                                       |
| 既定値 | なし                                                             |
| 基準  | ワークスペースルート                                                     |
| 検証  | 空不可                                                            |
| 役割  | build 結果のルート。 <code>serve</code> / <code>dev</code> の公開ルートでもある |

注意点:

- `build.clean` が `true` のとき、build 前に削除されることがあります
- 手編集した output は次の build で上書きされることがあります

- `build --all` では、複数 project の `outputDir` が同一または入れ子だと開始前に中断します

## files

推奨の files pipeline 設定です。配列で、各 entry は次のフィールドを持ちます。

| field   | 基準                     | 必須 | 既定                    | 説明                  |
|---------|------------------------|----|-----------------------|---------------------|
| from    | <code>sourceDir</code> | 必須 | —                     | 探索開始ディレクトリ          |
| to      | <code>outputDir</code> | 任意 | <code>''</code>       | 出力基準。空なら output ルート |
| include | from                   | 任意 | <code>['**/*']</code> | 取り込み glob           |
| exclude | from                   | 任意 | <code>[]</code>       | 除外 glob             |

例:

```
files: [
  {
    from: 'pages',
    to: '',
  },
]
```

- 複数 entry を書けます
- 同じ output path が複数 source から生成されるとエラーです
- `files` を空配列にすると files mode にはなりません (legacy 判定側へ)
- `files` と `render` / `copy` の同時設定はできません

処理の詳細は [files pipeline](#) を参照してください。

## templates

| 項目  | 内容                      |
|-----|-------------------------|
| 型   | string の配列              |
| 必須  | 任意                      |
| 既定値 | <code>[]</code>         |
| 基準  | <code>sourceDir</code>  |
| 役割  | Nunjucks loader の追加検索パス |

例:

```
templates: ['layouts', 'components'],
```

- `extends` / `include` の解決に使います
- 実在するディレクトリ配下は、files mode では **直接 output しません**
- `sourceDir` 自体も loader root に含まれます

## data

| 項目  | 内容                                    |
|-----|---------------------------------------|
| 型   | plain object ( <code>null</code> 可)   |
| 必須  | 任意                                    |
| 既定値 | 空に近い扱い (マージ結果)                        |
| 役割  | すべての Nunjucks render context に渡す共通データ |

公式 sample では `site.name`などを渡しています。

### Warning

`data.rootPath` は予約語です。設定するとエラーになります。  
`rootPath` は JSKim が各ページへ自動注入します。

## build 設定

| path                     | 型       | 既定                | 説明                                    |
|--------------------------|---------|-------------------|---------------------------------------|
| <code>build.clean</code> | boolean | <code>true</code> | build 前に <code>outputDir</code> を削除する |

## watch 設定

| path                        | 型             | 既定               | 説明                           |
|-----------------------------|---------------|------------------|------------------------------|
| <code>watch.debounce</code> | number (0 以上) | <code>150</code> | 最後の変更から再 build までの待ち時間 (ミリ秒) |

## serve 設定

| path                    | 型            | 既定                       | 説明        |
|-------------------------|--------------|--------------------------|-----------|
| <code>serve.host</code> | string (空不可) | <code>'127.0.0.1'</code> | 待ち受け host |
| <code>serve.port</code> | 整数 1~65535   | <code>3000</code>        | 待ち受け port |

CLI の `--host` / `--port` がある場合は、CLI が優先されます。

## dev 設定

| path                        | 型       | 既定                | 説明                      |
|-----------------------------|---------|-------------------|-------------------------|
| <code>dev.liveReload</code> | boolean | <code>true</code> | 開発中の live reload を有効にする |

## filters と globals

カスタム Nunjucks 機能は `nunjucks` 配下に入ります。

```
nunjucks: {
  filters: {
    formatPrice(value) {
      return `${Number(value).toLocaleString('ja-JP')}円`;
    },
  },
  globals: {
    currentYear() {
      return new Date().getFullYear();
    },
  },
}
```

- `filters` の値は **function** である必要があります
- JavaScript の関数として定義します (JSON では書けません)
- 公式 sample はデモ用に定義していますが、必須ではありません
- 非同期 filter は現在サポートしていません

書き方の詳細は [Nunjucksの使い方](#) を参照してください。

## 全体 option reference

### トップレベル

| path                  | 型      | 必須 | 既定              | 説明         |
|-----------------------|--------|----|-----------------|------------|
| <code>projects</code> | object | 必須 | —               | project 定義 |
| <code>defaults</code> | object | 任意 | <code>{}</code> | 共通既定値      |

### project / defaults で使う主な key

| path                   | 型      | 必須 | 既定 | 説明         |
|------------------------|--------|----|----|------------|
| <code>sourceDir</code> | string | 必須 | —  | source ルート |

| path                          | 型              | 必須       | 既定                       | 説明                |
|-------------------------------|----------------|----------|--------------------------|-------------------|
| <code>outputDir</code>        | string         | 必須       | —                        | output ルート        |
| <code>files</code>            | object[]   未設定 | mode 依存  | 未設定は <code>null</code>   | files pipeline    |
| <code>templates</code>        | string[]       | 任意       | <code>[]</code>          | template 検索パス     |
| <code>render</code>           | object[]       | legacy 時 | <code>[]</code>          | legacy render     |
| <code>copy</code>             | object[]       | 任意       | <code>[]</code>          | legacy copy       |
| <code>data</code>             | object   null  | 任意       | —                        | 共通 template データ   |
| <code>nunjucks.filters</code> | object         | 任意       | <code>{}</code>          | カスタム filter       |
| <code>nunjucks.globals</code> | object         | 任意       | <code>{}</code>          | カスタム global       |
| <code>build.clean</code>      | boolean        | 任意       | <code>true</code>        | output を clean    |
| <code>watch.debounce</code>   | number         | 任意       | <code>150</code>         | 監視の debounce (ms) |
| <code>serve.host</code>       | string         | 任意       | <code>'127.0.0.1'</code> | サーバー host         |
| <code>serve.port</code>       | number         | 任意       | <code>3000</code>        | サーバー port         |
| <code>dev.liveReload</code>   | boolean        | 任意       | <code>true</code>        | live reload       |

legacy の `render` / `copy` 詳細は [docs/configuration.md](https://jskim.github.io/docs/configuration.md) も参照してください。  
 新規 project では `files` を推奨します。

## 検証と unknown key

主な検証例:

- 必須 key の欠落
- 型不正 (例: `serve.port` が範囲外)
- `files` と `render` / `copy` の同時設定
- `data.rootPath` の予約衝突
- path traversal ( `sourceDir` / `outputDir` の外へ出る設定 )

認識していない余分な key は、現在の実装ではエラーにせず無視されることがあります。  
 厳密な schema エンジンではありません。

エラー時の具体メッセージと復旧手順は [エラーとトラブルシューティング](#) を参照してください。

## 設定変更の反映

| command                               | config の読み方         |
|---------------------------------------|---------------------|
| <code>build</code>                    | 開始時に 1 回            |
| <code>serve</code>                    | 開始時に 1 回（監視しない）     |
| <code>watch</code> / <code>dev</code> | 開始時に読み、実行中の変更も検知できる |

`dev` では、次の値が変わると process 再起動が必要です。

- `outputDir`
- `serve.host`
- `serve.port`
- `dev.liveReload`

CLI で指定した `--host` / `--port` は、reload 後も維持されます。

不正な config を書いた場合は、直前まで有効だった設定を維持する動作があります。

詳細な hot reload の挙動は [開発機能](#) を参照してください。

## 複数 project の例

`outputDir` が衝突しないように分けます。

```
module.exports = {
  defaults: {
    files: [{ from: 'pages', to: '' }],
    templates: ['layouts', 'components'],
    serve: { host: '127.0.0.1', port: 3000 },
  },
  projects: {
    siteA: {
      sourceDir: 'src/site-a',
      outputDir: 'dist/site-a',
    },
    siteB: {
      sourceDir: 'src/site-b',
      outputDir: 'dist/site-b',
    },
  },
};
```

```
jskim build siteA
jskim build --all
jskim dev siteB --port 4000
```

# files pipeline

この章では、JSKim が source を output に変換する files pipeline の規則を説明します。  
公式 sample の配置は [プロジェクト構成](#) もあわせて参照してください。

## pipeline の概要

推奨設定では、`files` の各 entry から処理計画を作ります。



`templates` に指定した実在ディレクトリ配下は、計画から除外され、直接 output されません。

## files entry

公式 sample の設定:

```
files: [
  {
    from: 'pages',
    to: '',
  },
]
```

| field         | 意味                    |
|---------------|-----------------------|
| from: 'pages' | sourceDir/pages を探索する |
| to: ''        | outputDir ルートへ出す      |

そのため、例えば次のように対応します。

```
src/sample/pages/index.html.njk
→ dist/sample/index.html
```

## render 対象

判定は **パス末尾が .njk (小文字) かどうか** です。

| source         | 処理                  |
|----------------|---------------------|
| index.html.njk | render → index.html |



| source        | 処理                   |
|---------------|----------------------|
| style.css.njk | render → style.css   |
| main.js.njk   | render → main.js     |
| common.css    | copy                 |
| logo.svg      | copy                 |
| note.NJK      | copy (末尾が .njk ではない) |

### Note

判定は case-sensitive です。 .NJK や .Njk は render しません。

ディレクトリ自体は出力対象ではなく、配下のファイルが対象です。

## output のファイル名

render 時は、末尾の .njk 4 文字だけ を外します。

```
index.html.njk → index.html
style.css.njk  → style.css
main.js.njk    → main.js
```

### Note

example.njk.njk のように二重になっている場合も、末尾の 1 回分だけ外します。  
結果のファイル名は example.njk になります。

## copy 対象

末尾が .njk でないファイルは byte copy します。

- 内容を変換しません
- 文字コード変換も行いません
- 公式 sample の平文 CSS や SVG がこれに当たります

例:

- [common.css](#)
- [logo.svg](#)

## directory 構造の維持

from 配下の相対パスは、to 基準で output 側にも維持されます。

```
pages/crud/detail.html.njk
→ dist/sample/crud/detail.html

pages/dashboard/assets/css/dashboard.css
→ dist/sample/dashboard/assets/css/dashboard.css
```

## template root

次の設定があるとき、`layouts` と `components` は template root です。

```
templates: [
  'layouts',
  'components',
]
```

用途:

- `{% extends "layouts/base.njk" %}`
- `{% include "components/header.njk" %}`

これらは配布用 HTML としては直接生成されません。

例: [base.njk](#)

## rootPath

各 render ページには、output 上の位置から `outputDir` ルートへ戻る相対パス `rootPath` が入ります。

計算の考え方:

1. output ファイルのディレクトリから `outputDir` への相対パスを取る
2. HTML 用に `/` 区切りにする
3. 空なら `./` にする（末尾に `/` を付ける）

公式 sample (`outputDir = dist/sample`) での例:

| output page             | rootPath |
|-------------------------|----------|
| index.html              | ./       |
| dashboard/index.html    | ../      |
| crud/detail.html        | ../      |
| さらに深い階層 (例: a/b/c.html) | ../../   |

共通 asset の参照例 (layout) :

```
<link rel="stylesheet" href="{{ rootPath }}assets/css/common.css">
```

`data.rootPath` で上書きすることはできません。

## 共通 asset と page-local asset

### 共通 asset

```
pages/assets/css/common.css
→ assets/css/common.css
```

深い階層のページからも同じ場所を指すため、`rootPath` を使います。

```
{{ rootPath }}assets/css/common.css
```

### page-local asset

```
pages/crud/assets/css/crud.css
→ crud/assets/css/crud.css
```

同じ画面グループからの相対パスで参照します。

```
<link rel="stylesheet" href="assets/css/crud.css">
```

#### Warning

page-local CSS に `rootPath` を付けしないでください。

```
{{ rootPath }}assets/css/crud.css
```

これは output root の `assets/css/crud.css` を指します。  
実際のファイルは `crud/assets/css/crud.css` にあるため、パスがずれます。

| 種類         | 置き場所                                        | 参照                                    |
|------------|---------------------------------------------|---------------------------------------|
| 共通         | <code>pages/assets/...</code>               | <code>{{ rootPath }}assets/...</code> |
| page-local | <code>pages/&lt;group&gt;/assets/...</code> | <code>assets/...</code> (相対)          |

実例:

- 共通: [common.css](#)
- Dashboard: [dashboard.css](#)
- CRUD: [crud.css](#)
- Wizard: [wizard.css](#)

## 複数の files entry

複数の entry を書くと、それぞれの計画が結合されます。

```
files: [
  { from: 'pages', to: '' },
  { from: 'shared-images', to: 'assets/img' },
]
```

別々の source を、異なる output 位置へ出せます。

ただし最終的な output path が同じになると衝突エラーになります。

## output の衝突

次のような場合、build は失敗します。

- 2 つの source が同じ output path を生成する
- `main.js` と `main.js.njk` のように、suffix 除去後に同じパスになる
- Windows で大文字小文字だけが違う path（同一扱いになり衝突）

衝突は、ファイルを書き始める前の計画段階で検出されます。

エラーメッセージの読み方は [エラーとトラブルシューティング](#) を参照してください。

## clean

`build.clean` の既定値は `true` です。

| 値                  | 動作                                         |
|--------------------|--------------------------------------------|
| <code>true</code>  | build 前に <code>outputDir</code> を削除してから再生成 |
| <code>false</code> | 既存ファイルを残したまま上書き生成                          |

`watch` / `dev` の再 build でも、`clean: true` なら毎回フル clean + 全体再 build です。

そのため、source から消したファイルは、次の成功 build で output から消えます。

危険な `outputDir`（ワークスペースルート全体など）への clean は拒否されます。

## watch 中の変更・削除

概要のみ示します。

| 変化     | 結果の考え方              |
|--------|---------------------|
| ファイル修正 | 再 build で output 更新 |
| 新規ファイル | 再 build で追加         |

| 変化                    | 結果の考え方                                                                   |
|-----------------------|--------------------------------------------------------------------------|
| 削除                    | <code>clean: true</code> なら output からも消える                                |
| layout / component 変更 | 依存ページも含めて再 build                                                         |
| config 変更             | <code>watch</code> / <code>dev</code> では検知できる（詳細は <a href="#">開発機能</a> ） |

debounce や live reload の詳細は [開発機能](#) を参照してください。

## 安全な asset 配置の目安

| 目的                       | 置き場所 / 命名                                                   |
|--------------------------|-------------------------------------------------------------|
| 全ページ共通                   | <code>pages/assets/</code>                                  |
| 特定画面群だけ                  | その画面群の <code>assets/</code>                                 |
| Nunjucks で生成したい CSS / JS | 末尾に <code>.njk</code> （例: <code>style.css.njk</code> ）      |
| そのままコピーしたいファイル           | 平文の拡張子（例: <code>common.css</code> , <code>logo.svg</code> ） |
| layout / component       | <code>templates</code> に指定した root                           |

## 公式 sample の対応表

| source                                                | 処理       | output                                          |
|-------------------------------------------------------|----------|-------------------------------------------------|
| <code>pages/index.html.njk</code>                     | render   | <code>index.html</code>                         |
| <code>pages/dashboard/index.html.njk</code>           | render   | <code>dashboard/index.html</code>               |
| <code>pages/dashboard/assets/css/dashboard.css</code> | copy     | <code>dashboard/assets/css/dashboard.css</code> |
| <code>pages/crud/detail.html.njk</code>               | render   | <code>crud/detail.html</code>                   |
| <code>pages/crud/assets/css/crud.css</code>           | copy     | <code>crud/assets/css/crud.css</code>           |
| <code>pages/wizard/assets/css/wizard.css</code>       | copy     | <code>wizard/assets/css/wizard.css</code>       |
| <code>pages/assets/css/common.css</code>              | copy     | <code>assets/css/common.css</code>              |
| <code>pages/assets/img/logo.svg</code>                | copy     | <code>assets/img/logo.svg</code>                |
| <code>layouts/base.njk</code>                         | template | 直接 output なし                                    |
| <code>components/header.njk</code>                    | template | 直接 output なし                                    |

設定の全体像は [設定](#)、コマンド操作は [CLIリファレンス](#) を参照してください。

# Nunjucksの使い方

この章では、JSKim の静的ページ制作で実用的な Nunjucks 文法を説明します。  
Nunjucks 全体の公式 reference を置き換えるものではありません。

## JSKim と Nunjucks

| 役割       | 内容                                                           |
|----------|--------------------------------------------------------------|
| Nunjucks | template の記法と render engine                                  |
| JSKim    | project / 設定 / files / build / watch / serve / dev をまとめる実行環境 |

`{% ... %}` と `{{ ... }}` は Nunjucks の文法です。  
一方、`rootPath` は Nunjucks 固有の変数ではなく、JSKim が render context に注入する予約値です。  
Nunjucks の機能すべてが JSKim 独自機能というわけではありません。  
この章は、公式 sample で使われている範囲を中心にします。

## Template inheritance

各ページは共通 layout を継承します。

```
{% extends "layouts/base.njk" %}
```

パスは、page file からの相対パスではありません。  
Nunjucks loader が探す template root (`sourceDir` と `templates[]`) 基準です。  
layout 自体は template root のため、直接 HTML としては output されません。  
詳細は [files.pipeline](#) を参照してください。

- layout: [base.njk](#)
- 例: [Portal](#)、[Dashboard](#)

## block

[base.njk](#) が定義する block は次の 3 つです。

| block                | 用途                             |
|----------------------|--------------------------------|
| <code>title</code>   | <code>&lt;title&gt;</code> の内容 |
| <code>pageCss</code> | 画面群ごとの stylesheet など           |
| <code>content</code> | 本文                             |

Wizard 入力ページの例:

```
{% block title %}情報入力 - {{ site.name }}{% endblock %}

{% block pageCss %}
  <link rel="stylesheet" href="assets/css/wizard.css">
{% endblock %}

{% block content %}
  ...
{% endblock %}
```

公式 sample に存在しない `script` block などここでは扱いません。

## include

layout は共通 fragment を `include` します。

```
{% include "components/header.njk" %}
{% include "components/sidebar.njk" %}
{% include "components/footer.njk" %}
```

breadcrumb は `breadcrumbs` があるときだけ読み込みます。

```
{% if breadcrumbs %}
  {% include "components/breadcrumb.njk" %}
{% endif %}
```

Wizard ページでは手順表示も include します。

```
{% include "components/wizard-steps.njk" %}
```

include 対象も独立 HTML としては output されません。

- [header.njk](#)
- [wizard-steps.njk](#)
- 例: [Wizard 入力](#)

## Variable output

config の `data` や page 内の値を出力します。

```
{{ site.name }}
```

JSKim の Nunjucks 環境は `autoescape: true` です。

HTML 特殊文字は既定で escape されます。

信頼できない入力を raw HTML として埋め込む例は、このガイドでは扱いません。

## set

page 内の静的な値を定義します。

```
{% set currentSection = "dashboard" %}
{% set currentStep = 1 %}
```

CRUD 一覧では、表示用の配列も page 内で定義しています。

```
{% set products = [
  { id: "P-001", name: "サンプル商品A", status: "公開" },
  ...
] %}
```

これは外部 API や mock server ではありません。

application の状態管理機能でもありません。静的 UI sample のための値です。

例: [CRUD 一覧](#)

## for

繰り返し HTML を生成します。CRUD 一覧の抜粋:

```
{% for product in products %}
<tr>
  <td>{{ product.id }}</td>
  <td>{{ product.name }}</td>
  ...
</tr>
{% endfor %}
```

空配列の特別処理など、sample にない高度な用法は省略します。

## if / elif / else

条件で class や表示文言を切り替えます。

CRUD の status 表示:

```
{% if product.status == "公開" %}
  <span class="badge badge--public">公開</span>
{% elif product.status == "非公開" %}
  <span class="badge badge--private">非公開</span>
{% else %}
  <span class="badge badge--draft">下書き</span>
{% endif %}
```



色だけで区別せず、テキストも残しています。

Wizard の手順表示でも `currentStep` を比較します。

```
{% if currentStep == 1 %} is-current{% elif currentStep > 1 %} is-complete{% endif %}
```

例: [wizard-steps.njk](#)

## default

公式 sample では `default` filter を使っていません。

Nunjucks 自体の機能として存在しますが、このガイドの代表例には含めません。

## Escaping

JSKim は環境作成時に次を固定しています。

- `autoescape: true`
- `noCache: true`

ユーザーが `jskim.config.js` の `nunjucks` で `autoescape` を切り替える設定はありません。  
`nunjucks` 配下で設定できるのは主に `filters` と `globals` です。

### Warning

信頼できる HTML だけを意図的に escape 解除したい場合は、  
filter 側で Nunjucks の `SafeString` を返す方法があります（公式 config の `toJson` 例）。  
不特定の入力に対して安易に escape を外さないでください。

XSS 対策の包括ガイドは、この章の範囲外です。

## rootPath

`rootPath` は Nunjucks 固有の組み込み変数ではありません。

JSKim が各ページの output 位置に応じて注入する予約 context です。

| output                            | rootPath         |
|-----------------------------------|------------------|
| <code>index.html</code>           | <code>./</code>  |
| <code>dashboard/index.html</code> | <code>../</code> |
| <code>crud/detail.html</code>     | <code>../</code> |

共通 asset の例:

```
{{ rootPath }}assets/css/common.css
```

## Warning

page-local asset には `rootPath` を付けないでください。  
詳細と正しい例は [files pipeline](#) を参照してください。

`data.rootPath` を設定すると予約語衝突エラーになります。

## Config data

`jskim.config.js` の `data` は、すべての render page の context に渡されます。

```
data: {
  site: {
    name: 'JSKim UI Sample',
    language: 'ja',
    themeColor: '#222222',
  },
}
```

`{{ site.name }}` のように nested object を参照できます。

| 種類                     | 違い                         |
|------------------------|----------------------------|
| <code>data</code>      | project 全体で共通。config に書く   |
| <code>{% set %}</code> | その page（または template）内の局所値 |

設定の詳細は [設定](#) を参照してください。

## filters と globals

登録場所:

```
nunjucks: {
  filters: { ... },
  globals: { ... },
}
```

- filter は **function** である必要があります
- global は function または通常値を登録できます
- 非同期 function はサポートしていません

公式 config にはデモ用の `formatPrice` / `toJson` / `currentYear` があります。

ただし公式 UI sample のページ本文は、これらに依存していません。

starter project の理解に必須ではありません。

## nunjucks 設定でできること

現在の schema でユーザーが書く主な key:

| path             | 内容          |
|------------------|-------------|
| nunjucks.filters | カスタム filter |
| nunjucks.globals | カスタム global |

JSKim が環境作成時に固定する値:

| option     | 値                           |
|------------|-----------------------------|
| autoescape | true                        |
| noCache    | true (loader / Environment) |

Nunjucks upstream のすべての Environment option が、そのまま config から指定できるわけではありません。

## 公式 sample の文法 mapping

| 文法           | sample file                                 | 用途                           |
|--------------|---------------------------------------------|------------------------------|
| extends      | <a href="#">pages/index.html.njk</a> など     | 共通 layout 継承                 |
| block        | <a href="#">layouts/base.njk</a>            | title / pageCss / content    |
| include      | <a href="#">layouts/base.njk</a>            | header / sidebar / footer など |
| set          | <a href="#">pages/crud/index.html.njk</a>   | 静的データ・現在セクション                |
| for          | <a href="#">pages/crud/index.html.njk</a>   | 一覧行の繰り返し                     |
| if           | <a href="#">pages/crud/index.html.njk</a>   | status badge 分岐              |
| include + if | <a href="#">components/wizard-steps.njk</a> | Wizard 手順表示                  |

## この章で扱わない範囲

次は公式 sample の範囲外、または別途の高度なトピックです。

- macro
- 非同期 filter
- custom extension
- Nunjucks JavaScript API 全体
- backend での動的 template 実行
- ブラウザが Nunjucks を実行する方式 (build 時に HTML へ展開されます)

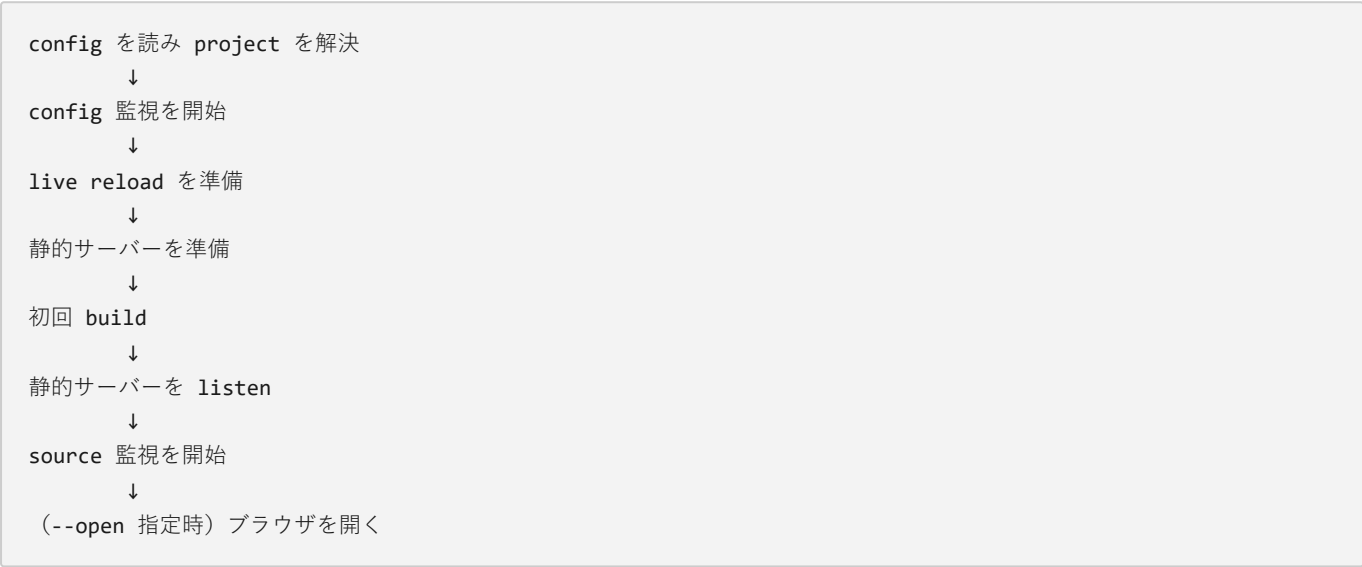
開発中のエラー表示や reload は [開発機能](#)、  
エラー対処は [エラーとトラブルシューティング](#) を参照してください。

# 開発機能

この章では、主に `dev` 実行中の開発支援機能を説明します。  
コマンドの選び方は [基本的な開発workflow](#) も参照してください。

## dev の構成

`dev` は次の順で起動します。



| command            | build | 監視  | サーバー | live reload |
|--------------------|-------|-----|------|-------------|
| <code>watch</code> | する    | する  | しない  | しない         |
| <code>serve</code> | しない   | しない | する   | しない         |
| <code>dev</code>   | する    | する  | する   | する（既定）      |

`dev.liveReload` を `false` にすると、live reload 注入と SSE は無効になります。

## Live reload

`dev` かつ `liveReload: true` のとき、ブラウザは開発サーバーと接続します。

- HTML レスポンスにだけクライアント script が注入されます（`dist` には書き込みません）
- source 変更後の再 build 成功に応じて、ブラウザへ通知します
- 接続が切れた場合、クライアントは再接続を試みます

すべての変更が page 全体の reload になるわけではありません。  
CSS の一部は soft reload になります。

内部エンドポイントは `/_jskim/live-reload` です。  
`serve` や `liveReload: false` では利用できません。

## Full reload

次のような変更では、通常 page 全体を再読み込みします。

- HTML / Nunjucks ( `.html.njk` など) の変更
- 画像や JS などの非 CSS 変更
- layout / component ( `templates[]` 配下) の変更
- CSS でも `add` / `unlink` など `change` 以外のイベント
- CSS soft reload が失敗したときの fallback
- config の再読み込み後に成功した再 build (概要)

## CSS soft reload

条件を満たすとき、page 全体を再読み込みせず、stylesheet だけを差し替えます。

### CSS soft reload

→ 現在の page を保ったまま stylesheet を読み直す

### Full reload

→ page 全体を読み直す

## 対象になる条件

再 build 成功時の変更イベントが **すべて** 次を満たす場合です。

- イベント種別が `change`
- 対象パスの末尾が `.css` または `.css.njk` (大文字小文字を区別しない)
- `templates[]` 配下ではない

公式 sample の page-local CSS (平文 `.css`) も対象になり得ます。

## ブラウザ側の制限

クライアントは次だけを更新します。

- `<link rel="stylesheet">` ( `rel` に `stylesheet` を含むもの)
- `same-origin` の URL

次は soft reload の対象外です。

- 外部 CDN の stylesheet
- `<style>` の inline CSS
- 動的に後から挿入した stylesheet (保証なし)
- `same-origin` の stylesheet link が 1 つもない場合 → full reload

読み込み失敗やタイムアウト時も full reload にフォールバックします。

## Note

form 入力内容やスクロール位置などの「状態保持」を、あらゆるケースで保証するものではありません。

## Browser error overlay

template / config / build のエラーを、開発中のブラウザ上に表示できます。

主な特徴:

- 現在表示中の page の上に重ねて表示
- Shadow DOM を使い、page の CSS から隔離
- メッセージはテキストとして表示（タイトルは `JSKim dev error`）
- `Close` ボタンで一時的に閉じられる
- エラーが解消され、成功通知を受けると自動で消える

stack trace 全体をそのまま出す UI ではありません。

サーバーが送るメッセージ本文が中心です。

## Last error replay

ブラウザがエラー発生後に接続した場合や、再接続した直後でも、まだエラー状態が残っていれば、サーバーが最後のエラーを再送します。

複数のブラウザクライアントが接続している場合、同じ通知を受け取れます。

## Recovery

エラーを直して再 build が成功すると、おおむね次の流れになります。

1. サーバー側のエラー状態が解除される
2. overlay が消える（`clear-error`）
3. 変更種別に応じて full reload または CSS soft reload

config エラー中は、source 側の成功だけでは overlay を消さない優先順位があります。

config を直して有効な設定に戻す必要があります。

process 自体は、個別の再 build 失敗だけでは終了しません。

## Config hot reload

| command | jskim.config.js を監視するか |
|---------|------------------------|
| build   | しない                    |

| command | jskim.config.js を監視するか |
|---------|------------------------|
| serve   | しない                    |
| watch   | する                     |
| dev     | する                     |

読み込みや検証に失敗した場合:

- 直前まで有効だった設定 (last-known-good) を継続
- terminal にエラーを表示
- `dev` では browser overlay にも表示できる

一時的に設定ファイルが消えた場合も、以前の正常な設定を継続します。

成功して再読み込みできた場合は、監視対象を更新し、再 build します。

## Restart-required 設定

`dev` では、次の値が変わると **process の再起動が必要** です。

- `outputDir`
- `serve.host`
- `serve.port`
- `dev.liveReload`

この場合、warning を出し、以前の正常な設定を継続します。

process を自動終了はしません。手動で `dev` を再起動してください。

それ以外の多くの設定は、hot reload で反映を試みます。

## CLI override の維持

例:

```
jskim dev sample --host 127.0.0.1 --port 4000 --open
```

- `--host` / `--port` は config hot reload 後も CLI 指定が優先されます
- `--open` は起動時に **1 回だけ** ブラウザを開きます (reload のたびに開き直しません)

## Browser open

`dev --open` は、サーバーの listen 成功後に OS の既定ブラウザで URL を開きます。

失敗した場合:



- warning を出す
- 開発サーバー自体は継続する

手動で次の URL を開けば確認できます（既定時）。

```
http://127.0.0.1:3000/
```

### Note

`serve` に `--open` はありません。

## Strict CSP について

JSKim は live reload 用 script を HTML レスポンスへ注入します。  
overlay は Shadow DOM を使います。

Strict な Content-Security-Policy 環境で、注入 script や接続が制限される可能性があります。  
ただし、現時点の公式 test で CSP ごとの動作を保証してはいません。

## 開発機能の選び方

| 必要                            | 使い方                                 |
|-------------------------------|-------------------------------------|
| output だけ継続更新                 | <code>watch</code>                  |
| 既存 <code>dist</code> をブラウザで確認 | <code>serve</code>                  |
| reload 込みの統合開発                | <code>dev</code>                    |
| 起動時にブラウザを開く                   | <code>dev --open</code>             |
| CSS 変更時に page 全体 reload を減らす  | <code>dev</code> （条件付き soft reload） |

## この章に関連する制限

- JavaScript HMR ではありません
- JS module の実行状態保持は保証しません
- external / inline stylesheet は soft reload 対象外です
- browser / OS 環境に依存する部分があります（`--open` など）

全体の製品制限事項は [制限事項](#) を参照してください。

エラー別の対処は [エラーとトラブルシューティング](#) を参照してください。

# エラーとトラブルシューティング

この章では、よくある問題の症状、原因、JSKim の動作、確認方法、解決の手がかりを整理します。  
CLI の構文は [CLIリファレンス](#)、設定は [設定](#) も参照してください。

## エラー処理の概要

| 状況                    | build  | watch               | serve   | dev                    |
|-----------------------|--------|---------------------|---------|------------------------|
| template / render エラー | 終了（失敗） | process 維持          | —       | process 維持 + overlay 可 |
| config エラー（起動時）       | 終了（失敗） | 終了（失敗）              | 終了（失敗）  | 終了（失敗）                 |
| config エラー（実行中の再読込）   | —      | last-known-good を維持 | —       | 維持 + overlay 可         |
| port 衝突               | —      | —                   | 起動失敗で終了 | 起動失敗で終了                |

— はその command では通常起きない、または直接関係しないことを表します。

## 問題を調べる基本の順序

1. terminal のメッセージを読む
2. `dev` なら browser overlay も確認する
3. 直前に編集した file を見直す
4. `jskim.config.js` と path を確認する
5. port / 別 process の占有を確認する
6. restart-required の設定変更なら command を再起動する

いきなり依存関係の再インストールやキャッシュ削除から始めないでください。

## Unknown command / option

### 症状

コマンドや option が認識されない。

### 原因

- 存在しない command 名
- その command で使えない option
- `--port=4000` のような `=` 付き記法

- `--` 単独トークン
- option の重複や値の欠落

## JSKim の動作

引数解析の段階で失敗し、process は終了します (exit code `1`)。

代表メッセージ:

```
[JSKim] 不明なコマンドです: <command>
```

```
[JSKim] 不明なoptionです: <token>
```

## 確認・解決

- `jskim --help` で使える command を確認する
- `serve` に `--open` は無い (`dev --open` を使う)
- `--port 4000` のようにスペース区切りで書く

## Project 選択のエラー

### 症状

どの project を対象にするか決められない。

### 原因と動作

| 状況                                  | 結果         |
|-------------------------------------|------------|
| <code>projects</code> が 0 件         | エラー        |
| 1 件だけ                               | 名前省略可      |
| 2 件以上で省略                            | エラー (候補一覧) |
| 存在しない名前                             | エラー        |
| project 名と <code>--all</code> の同時指定 | エラー        |

代表メッセージ:

```
[JSKim] projectを指定してください。
```

## 解決例

```
jskim build sample
jskim build --all
```

## Config file の読み込みエラー

### 症状

起動直後に設定を読めない。

### 原因の例

- ワークスペースに `jskim.config.js` が無い
- JavaScript の構文エラー
- `module.exports` がオブジェクトでない
- ESM (`export default`) 形式を使っている
- `require` 時の例外

代表メッセージ:

```
[JSKim] 設定ファイルが見つかりません: jskim.config.js
```

### 解決

- コマンドを project ルートで実行する
- CommonJS (`module.exports = { ... }`) で書く
- 構文エラーを修正する

## Config 検証エラー

### 症状

設定は読めたが、値が不正。

### よくある原因

- `projects` が無い / オブジェクトでない
- `sourceDir` / `outputDir` の欠落や未存在
- `files` と `render` / `copy` の同時設定
- `serve.port` / `watch.debounce` の型や範囲不正
- `data.rootPath` (予約語衝突)

#### Note

認識していない余分な key は、必ずしもエラーになりません。  
無視されることがあります。

## 解決

[設定](#) の型と既定値を確認し、問題の key だけ直します。

## Template / render エラー

### 症状

Nunjucks の構文ミス、見つからない `extends` / `include`、filter 実行時エラーなど。

### JSKim の動作

| command                               | 動作                                                      |
|---------------------------------------|---------------------------------------------------------|
| <code>build</code>                    | 失敗して終了 (exit code <code>1</code> )                      |
| <code>watch</code> / <code>dev</code> | terminal に表示。process は維持。 <code>dev</code> では overlay 可 |

代表的な見出し:

```
[JSKim] プロジェクト "<name>" の Nunjucks レンダリングに失敗しました。
```

### 確認・解決

1. 表示されたソース / テンプレート path を開く
2. `extends` / `include` のパスが template root 基準か確認する
3. 修正して保存する (`watch` / `dev` なら再 build)
4. 成功すれば overlay は消える

文法の基本は [Nunjucksの使い方](#) を参照してください。

## Output パスの衝突

### 症状

同じ output を複数の source が生成しようとする。

### 原因の例

- 複数の `files` entry が同じ出力先に重なる
- `main.js` と `main.js.njk` のように suffix 除去後に同名になる
- Windows で大文字小文字だけが違う path

### JSKim の動作

書き込み前の計画段階で検出し、build は失敗します。

## 解決

- `from` / `to` / ファイル名を分ける
- どちらかを `exclude` する
- 命名規則を揃える

詳細は [files pipeline](#) を参照してください。

## 複数 project の outputDir 衝突

### 症状

`jskim build --all` が開始前に止まる。

### 原因

- 同じ `outputDir`
- 入れ子（祖先 / 子孫）の `outputDir`

これは files 内の output 衝突とは別です。project 間の互換性検査です。

## 解決

各 project の `outputDir` を重ならない場所に分けます。

```
projects: {
  siteA: { sourceDir: 'src/a', outputDir: 'dist/a' },
  siteB: { sourceDir: 'src/b', outputDir: 'dist/b' },
}
```

## Port already in use

### 症状

`serve` / `dev` が待ち受けできない。

### 原因

既定の `3000`（または指定 port）を他 process が使用中（`EADDRINUSE`）。

代表メッセージ:

```
[JSKim] ポート <port> はすでに使用されています。
```

## JSKim の動作

自動で次の空き port を探しません。起動に失敗した process は終了します。

## 解決例

```
jskim dev sample --port 3001
```

または `jskim.config.js` の `serve.port` を変更します。  
占有している process を止める方法は OS によって異なります。

## 不正な host / port

### 症状

CLI または config の待ち受け設定が拒否される。

### 原因の例

- `--port` の値欠落
- `0` / `65536` / 小数 / 非数値
- 空の host

検出は CLI 解析または設定検証の段階です。

### 正しい例

```
jskim serve sample --host 127.0.0.1 --port 4000
```

## Browser open の失敗

### 症状

`dev --open` でブラウザが開かない。

### JSKim の動作

warning を出し、**開発サーバーは継続**します。process 失敗にはしません。

### 解決

terminal に表示された URL を手動で開きます。

```
http://127.0.0.1:3000/
```

## CSS 変更が soft reload されない

### 確認順

1. `dev` で `liveReload: true` か
2. 変更が `.css` / `.css.njk` の `change` だけか
3. `templates[]` 配下の変更ではないか
4. page の `<link rel="stylesheet">` が same-origin か
5. href が実際の output を指しているか

よくある誤り (page-local) :

```
{{ rootPath }}assets/css/crud.css
```

正しい例:

```
<link rel="stylesheet" href="assets/css/crud.css">
```

条件を満たさない、または読み込みに失敗すると full reload になります。

詳細は [開発機能](#) を参照してください。

## Browser overlay が表示されない

### 確認

- `jskim dev` (または `npm run dev`) で起動しているか
- `dev.liveReload` が `true` か
- terminal にはエラーが出ているか
- ブラウザが live reload に接続できているか (接続再試行のログ)
- Strict CSP など注入 script が遮断されていないか (環境依存)

`serve` だけでは overlay は出ません。

## Config の修正が反映されない

### 確認

1. `watch` / `dev` で動かしているか (`build` / `serve` は監視しない)
2. 構文エラーや検証エラーで last-known-good のままになっていないか
3. restart-required (`outputDir` / `serve.host` / `serve.port` / `dev.liveReload`) ではないか
4. CLI の `--host` / `--port` が config より優先されていないか
5. terminal の warning を読んでいないか



必要なら `command` を止めて再起動します。

## build --all の一部失敗

### 症状

ログでは成功と失敗が混在し、最終的に失敗終了する。

### JSKim の動作

- 定義順に各 project を試す
- 途中失敗があっても次の project へ進む
- 1 件でも失敗があれば最終 exit code は `1`
- 成功した project の output は残ることがある

Warning

途中の成功ログだけを見て、全体が成功したと誤解しないでください。

## 削除した source が output に残る

### 原因と動作

| build.clean | 削除した source の扱い                                                |
|-------------|----------------------------------------------------------------|
| true (既定)   | 再 build 前に <code>outputDir</code> を消すため、消えたファイルも output から無くなる |
| false       | 上書き生成のみ。消えた source に対応する古い output が残ることがある                     |

`watch` / `dev` の再 build でも同様です。

### 解決

- 通常は `build.clean: true` を維持する
- `false` にしている場合は、不要ファイルを手動で整理するか、一度 clean 付き build を行う

## dist を直接編集したら消えた

### 原因

`dist/` は build の output です。次の clean / build で上書き・削除されることがあります。

## 解決

編集するのは `src/` (source) 側です。

見た目の確認用に `dist` を一時的に開くことはできますが、恒久的な修正場所にはしません。

## Internal link / asset が 404

### 確認

- source → output の対応 ([files pipeline](#))
- 共通 asset は `{{ rootPath }}assets/...`
- page-local は `assets/...`
- 先頭 `/` の絶対パスが意図どおりか (serve の root は `outputDir`)
- Windows 以外も含め、ファイル名の大文字小文字
- そもそも build 済みか

公式 sample の例:

- 共通: `assets/css/common.css`
- CRUD: `crud/assets/css/crud.css`

## Exit code と process 維持の要約

| 状況                                 | process  | 結果                                    |
|------------------------------------|----------|---------------------------------------|
| build 成功                           | 終了       | exit 0                                |
| build 失敗                           | 終了       | exit 1                                |
| build <code>--all</code> で 1 件以上失敗 | 終了       | exit 1                                |
| watch / dev の再 build 失敗            | 維持       | エラー表示 ( <code>dev</code> は overlay 可) |
| serve / dev の port 衝突              | 起動失敗側は終了 | exit 1                                |
| dev <code>--open</code> の失敗        | 維持       | warning                               |
| Ctrl+C での停止                        | 終了       | exit 0                                |

## 問い合わせ・報告に用意するとよい情報

- JSKim version (`jskim --version`)
- Node.js version
- package manager (npm / pnpm / yarn)
- OS
- 実行した command

- terminal のエラー全文（必要な範囲）
- `jskim.config.js` の関連部分
- 再現できる最小の directory 構成

### Note

報告前に、秘密情報、個人情報、社内専用 URL を取り除いてください。

公式の issue 先は、パッケージ / repository の案内に従ってください。  
(例: GitHub repository の Issues)

# Dashboard例

---

この章では、公式 Dashboard sample を通じて、共通 layout、component、template 内データ、共通 / page-local asset の使い方を確認します。

## Note

以下のコードは核心部分だけの抜粋です。  
全体の source は各 link から確認できます。

## Sample の目的

Dashboard は、管理画面でよく見かける構成を、1 画面にまとめた静的 UI sample です。

- 共通 layout / component / page の関係を確認できる
- summary card、table、notice などの見た目を確認できる
- 実際のデータ取得や chart 機能はない
- 表示値は template 内の静的値

Source:

- [pages/dashboard/index.html.njk](#)

## 関連 file

```
src/sample/  
├ layouts/base.njk  
├ components/  
│   ├── header.njk  
│   ├── sidebar.njk  
│   ├── breadcrumb.njk  
│   └── footer.njk  
└ pages/  
    ├── assets/css/common.css  
    └── dashboard/  
        ├── index.html.njk  
        └── assets/css/dashboard.css
```

共通 layout の詳細は [プロジェクト構成](#) と [Nunjucksの使い方](#) も参照してください。

## Page 構成

実際の Dashboard には次があります。

| 領域           | 内容                             |
|--------------|--------------------------------|
| page heading | 「Dashboard」と説明文                |
| breadcrumb   | Portal → Dashboard             |
| summary card | 4 枚（登録商品数 / 公開中 / 非公開 / 今月の更新） |
| 最近の更新 table  | 5 行の静的データと status badge        |
| お知らせ         | notice リスト                     |
| 画面ナビ         | Portal / CRUD / Wizard への移動    |

## Layout 継承

```
{% extends "layouts/base.njk" %}
```

- 共通の HTML document 構造を使う
- header / sidebar / footer は layout 側で include
- page は `title` / `pageCss` / `content` block を埋める
- layout 自体は直接 output されない

Layout:

- [layouts/base.njk](#)

## Page 変数

```
{% set currentSection = "dashboard" %}
{% set breadcrumbs = [
  { label: "Portal", href: rootPath + "index.html" },
  { label: "Dashboard" }
] %}
```

| 変数                          | 役割                                                                     |
|-----------------------------|------------------------------------------------------------------------|
| <code>currentSection</code> | sidebar の現在位置表示                                                        |
| <code>breadcrumbs</code>    | breadcrumb component の表示データ                                            |
| <code>block title</code>    | <code>&lt;title&gt;</code> （Dashboard - <code>{{ site.name }}</code> ） |

## Summary と table

summary と最近の更新は、template 内の配列です。

```
{% set summaryCards = [
  { label: "登録商品数", value: "120" },
  { label: "公開中", value: "98" },
  { label: "非公開", value: "22" },
  { label: "今月の更新", value: "15" }
] %}

{% for card in summaryCards %}
  <article class="summary-card">
    <p class="summary-card__label">{{ card.label }}</p>
    <p class="summary-card__value">{{ card.value }}</p>
  </article>
{% endfor %}
```

- 外部 API の応答ではない
- 繰り返す HTML を短く書く例
- 実際の application では、backend や build 前のデータ生成など、別の仕組みが必要

JSKim 自体は API を提供しません。

## Status 分岐

```
{% if item.status == "公開" %}
  <span class="badge badge--public">公開</span>
{% elif item.status == "非公開" %}
  <span class="badge badge--private">非公開</span>
{% else %}
  <span class="badge badge--draft">下書き</span>
{% endif %}
```

class とテキストの両方で状態を示します。色だけに頼らない書き方です。

## Asset

共通 CSS (layout) :

```
{{ rootPath }}assets/css/common.css
```

Dashboard CSS (page) :

```
<link rel="stylesheet" href="assets/css/dashboard.css">
```

| source                      | 処理   | output                |
|-----------------------------|------|-----------------------|
| pages/assets/css/common.css | copy | assets/css/common.css |

| source                                    | 処理   | output                             |
|-------------------------------------------|------|------------------------------------|
| pages/dashboard/assets/css/dashboards.css | copy | dashboard/assets/css/dashboard.css |

rootPath の詳細は [files.pipeline](#) を参照してください。

## Build output

```
dist/sample/dashboard/index.html
dist/sample/dashboard/assets/css/dashboard.css
dist/sample/assets/css/common.css
```

## 修正例

| 変えたいもの                  | 編集先                                           |
|-------------------------|-----------------------------------------------|
| summary の label / value | pages/dashboard/index.html.njk の summaryCards |
| table の行                | 同 file の recentUpdates                        |
| お知らせ文                   | 同 file の notices                              |
| Dashboard 専用の見た目        | pages/dashboard/assets/css/dashboard.css      |

保存後、`npm run dev` で確認できます。

## 新しい Dashboard page への拡張

最小の手順例:

1. 新しい `.html.njk` を `pages/` 配下を作る（または既存 page を参考にする）
2. `{% extends "layouts/base.njk" %}` する
3. `currentSection` / `breadcrumbs` を設定する
4. `content` block を書く
5. 必要なら page-local CSS を置く
6. `dev` または `build` で確認する

自動 generator や専用のコピーコマンドはありません。

## Sample の限界

- 実際の統計計算はない
- API / database はない
- chart library はない

- 認証はない
- 値は静的 sample

製品としての制限は [制限事項](#) を参照してください。

類似の画面構成として [CRUD例](#) と [Wizard例](#) もあります。



# CRUD例

---

この章では、公式 CRUD sample（商品管理の 6 画面）の構造と、静的な画面間 navigation を説明します。

## Warning

これは静的 UI sample です。  
JSKim が実際の登録・更新・削除機能を提供するわけではありません。

## Note

以下のコードは核心部分だけの抜粋です。  
全体の source は各 link から確認できます。

## Sample の目的

- 一覧 / 詳細 / 登録 / 編集 / 削除確認 / 完了という一般的な画面構成を示す
- page 単位の静的 HTML として確認できる
- 実際の登録・修正・削除は行わない
- 入力値の保存や画面間引き渡しはない

代表 source:

- [商品一覧](#)

## 関連 file

```
pages/crud/  
├ index.html.njk  
├ detail.html.njk  
├ create.html.njk  
├ edit.html.njk  
├ delete.html.njk  
├ complete.html.njk  
└ assets/css/crud.css
```

各 page:

- [一覧](#)
- [詳細](#)
- [登録](#)
- [編集](#)
- [削除確認](#)

- [完了](#)

共通 layout / component の詳細は [Dashboard例](#) と [Nunjucksの使い方](#) を参照してください。

## 画面の移動

```
商品一覧
└─ 商品詳細
  │ └─ 商品編集 → 処理完了
  │ └─ 商品削除確認 → 処理完了
  └─ 商品登録 → 処理完了

処理完了
└─ 商品一覧へ戻る
└─ Portal へ戻る
```

主な戻り先:

| 画面   | 戻るリンク       |
|------|-------------|
| 詳細   | 一覧          |
| 登録   | 一覧          |
| 編集   | 詳細          |
| 削除確認 | 詳細（削除せずに戻る） |
| 完了   | 一覧 / Portal |

## 商品一覧

template 内の `products` 配列を `for` で表示します。

```
{% for product in products %}
  <tr>
    <td>{{ product.id }}</td>
    <td><a href="detail.html">{{ product.name }}</a></td>
    ...
  </tr>
{% endfor %}
```

- table に caption / `scope="col"` がある
- 商品名と「詳細」はどちらも `detail.html` へ
- 「商品を登録する」は `create.html` へ
- status は `if` で badge を切替

## 商品詳細

単一の `product` object を page 内で定義し、表示します。

```
{% set product = {  
  id: "P-001",  
  name: "サンプル商品A",  
  ...  
} %}
```

- URL パラメータに応じた動的な取得ではない
- 編集 / 削除 / 一覧への navigation がある

## 商品登録

form の見た目だけを持つ画面です。

| 項目                     | 属性の例                               |
|------------------------|------------------------------------|
| 商品ID / 商品名 / カテゴリ / 価格 | <code>readonly</code> の text input |
| ステータス                  | <code>disabled</code> の select     |
| 商品説明                   | <code>readonly</code> の textarea   |

- 実際の submit はない
- 「登録する」は `complete.html` への静的リンク
- validation も入力保存もない

## 商品編集

登録画面に近い構成で、静的な初期値が入っています。

- 詳細画面の固定商品を編集する体裁
- 「更新する」も `complete.html` への静的リンク
- 実際の更新処理はない

### Note

登録と編集で form を macro / include にまとめていません。  
静的 UI sample として、画面ごとの構成を追いやすくするためです。

## 商品削除確認

削除対象の静的情報と注意文を示します。

- heading / 本文で「削除は実行されない」ことを明示
- 「削除する」は色付きボタンだが、実際の削除は行わず `complete.html` へ移動

## 処理完了

登録・更新・削除の流れから共通で使う静的な完了 page です。

- どの操作が成功したかを判定しない
- 一覧または Portal へ戻れる

## Nunjucks 文法 mapping

| 文法              | file                            | 用途           |
|-----------------|---------------------------------|--------------|
| set array       | <code>index.html.njk</code>     | 商品一覧         |
| for             | <code>index.html.njk</code>     | table row    |
| if              | <code>index.html.njk</code>     | status badge |
| set object      | <code>detail.html.njk</code> など | 静的な商品        |
| variable output | 複数 page                         | 商品情報の表示      |

基礎文法は [Nunjucksの使い方](#) を参照してください。

## CRUD CSS

| 項目     | パス                                                |
|--------|---------------------------------------------------|
| source | <code>pages/crud/assets/css/crud.css</code>       |
| output | <code>dist/sample/crud/assets/css/crud.css</code> |
| HTML   | <code>assets/css/crud.css</code>                  |

共通の見た目は `common.css`、CRUD 画面群の差分は `crud.css` が担当します。

## 新しい CRUD 画面への拡張

例:

- 一覧の column を増やす
- 詳細の項目を増やす
- 確認 page を分ける
- 完了 page を操作ごとに分ける
- CRUD 専用 CSS を拡張する

実際の backend 連携は JSKim の外（application 側）の作業です。

## Backend とつなぐ場合

概念だけ示します。

- JSKim の build 結果は静的ファイル
- API を呼ぶ JavaScript は利用者が別途書く
- server-side でデータを注入する framework ではない
- database / 認証 / 認可も JSKim の範囲外

特定の backend framework の実装例は、このガイドでは扱いません。

## Sample の限界

- 実際の CRUD 処理はない
- form submit はない
- validation はない
- persistence はない
- URL パラメータ別の詳細表示はない
- 外部 mock data file はない

詳細は [制限事項](#) を参照してください。

# Wizard例

---

この章では、公式 Wizard sample（3 ステップ）と共通の step component を説明します。

## Warning

入力内容の保存、画面間の引き継ぎ、送信処理はありません。  
確認画面の値も入力画面から渡されたものではありません。

## Note

以下のコードは核心部分だけの抜粋です。  
全体の source は各 link から確認できます。

## Sample の目的

- 情報入力 → 入力内容確認 → 完了という段階型の画面構成を示す
- layout / component / page-local CSS の使い方を確認する
- 実際の入力値の受け渡しや保存はしない

代表 source:

- [情報入力](#)

## 関連 file

```
components/wizard-steps.njk

pages/wizard/
├ input.html.njk
├ confirm.html.njk
├ complete.html.njk
└ assets/css/wizard.css
```

- [wizard-steps.njk](#)
- [入力](#)
- [確認](#)
- [完了](#)

共通 layout は [Dashboard例](#) と同じです。

## 画面の移動

情報入力  
↓  
入力内容確認  
↓  
完了

戻り:

入力内容確認 → 情報入力  
完了 → 情報入力 / Portal

「送信する」も完了画面への静的な移動です。

## currentStep

各 page が静的にステップ番号を持ちます。

| page   | 値                                      |
|--------|----------------------------------------|
| 情報入力   | <code>{% set currentStep = 1 %}</code> |
| 入力内容確認 | <code>{% set currentStep = 2 %}</code> |
| 完了     | <code>{% set currentStep = 3 %}</code> |

- browser / session の application state ではない
- `wizard-steps` component の表示切替のための値

## wizard-steps component

```
<li
  class="wizard-steps__item{% if currentStep == 1 %} is-current{% elif currentStep > 1 %} is-complete{% endif %}"
  {% if currentStep == 1 %}aria-current="step"{% endif %}
>
  <span class="wizard-steps__number">1</span>
  <span class="wizard-steps__label">情報入力</span>
  ...
</li>
```

- ステップは 3 つ
- 現在 / 完了 / 未実施を class とテキストで表現
- 現在ステップには `aria-current="step"`
- component 自体は直接 output されない

## 情報入力

見た目だけの入力欄です。

| 項目       | name / 属性                   |
|----------|-----------------------------|
| 氏名       | name / readonly             |
| メールアドレス  | email / readonly            |
| 電話番号     | phone / readonly            |
| お問い合わせ種別 | category / disabled select  |
| お問い合わせ内容 | message / readonly textarea |

- 実際の submit はない
- 「入力内容を確認する」は `confirm.html` へのリンク
- 入力内容は保存されない

## 入力内容確認

template 内の固定 object を表示します。

```
{% set application = {  
  name: "山田 太郎",  
  email: "taro@example.com",  
  phone: "090-0000-0000",  
  category: "商品について",  
  message: "sampleのお問い合わせ内容です。"  
} %}
```

### Note

`taro@example.com` は予約された example domain の例です。  
入力画面の内容が引き継がれた結果ではありません。

- 「入力画面へ戻る」→ `input.html`
- 「送信する」→ `complete.html` (実送信なし)

## 完了

静的な完了状態を示す画面です。

- 入力の送信・保存は行われていないと明記
- 「最初から確認する」→ `input.html`
- Portal へも戻れる



## Wizard CSS

| 項目     | パス                                                                                  |
|--------|-------------------------------------------------------------------------------------|
| source | <a href="pages/wizard/assets/css/wizard.css">pages/wizard/assets/css/wizard.css</a> |
| output | dist/sample/wizard/assets/css/wizard.css                                            |
| HTML   | assets/css/wizard.css                                                               |

step indicator、form、確認リスト、完了パネルなどの見た目を担当します。

共通部分は `common.css` です。

## アクセシビリティ

実際の sample が備えるもの:

- 手順は `<ol>`
- 現在ステップに `aria-current="step"`
- 各入力に `label`
- 状態テキスト（現在 / 完了 / 未実施）
- 色だけに頼らない状態表現

これ以上のアクセシビリティ機能を、sample が保証するわけではありません。

## ステップを増やす方法

4 ステップにする場合の概念的な順序:

1. 新しい page (`.html.njk`) を作る
2. `wizard-steps.njk` に項目を足す
3. 各 page の `currentStep` を調整する
4. navigation の link を直す
5. CSS を確認する
6. `dev` / `build` で link を検査する

自動 routing や共有 state の機能はありません。

## 実際の Wizard にする場合

JSKim の外で必要になる典型例:

- 入力値の保存
- validation
- 画面間の state

- submit / API / backend
- 直接 URL アクセスの制御
- 完了後の state クリア

具体的な application 実装コードは、このガイドでは扱いません。

## Sample の限界

- 入力の引き継ぎはない
- 送信・保存はない
- step は page ごとの静的値
- session / localStorage による状態管理はない

製品全体の境界は [制限事項](#) を参照してください。

# 制限事項

---

この章では、JSKim が提供しないことと、その意味を整理します。  
ツールの境界を理解し、別途何が必要かを判断するための章です。

## 静的サイトの build 環境である

**提供しないもの:** server-side runtime、SSR framework、SPA framework。

**意味:** build 結果は静的な HTML / CSS / 画像などです。サーバー上で template を都度実行する製品ではありません。

**必要な場合:** SSR や SPA が必要なら、その目的に合う別のツールを検討します。

### Note

JavaScript file 自体の render / copy は可能です。  
ただし JSKim は JavaScript bundler ではありません（後述）。

## Backend 機能はない

**提供しないもの:**

- API server
- database
- 認証 / 認可
- session
- 業務ロジック
- 実際の CRUD
- Wizard の入力 state 管理

**意味:** 公式 sample の CRUD / Wizard も画面構成の例です。

**必要な場合:** backend とフロントエンド連携は、利用者が別途実装します。

画面例は [CRUD例](#) / [Wizard例](#) を参照してください。

## HTML の自動 import / migration はない

**提供しないもの:** 既存 HTML の自動解析、分割、変換。

**意味:** 既存サイトの移行は、利用者が project に合わせて手作業（または別ツール）で行います。  
JSKim core の責任範囲ではありません。

将来の提供をこのガイドで約束しません。

## Bundler ではない

提供しないもの:

- JavaScript module bundling
- npm 依存の解決を伴う bundle
- transpile
- minify
- tree shaking
- code splitting

意味: `.js` / `.js.njk` は files pipeline で copy または Nunjucks render ですが、webpack / Vite のような変換パイプラインではありません。

必要な場合: bundling が必要なら専用の bundler を併用します。

## CSS preprocessor ではない

提供しないもの: Sass / Less / PostCSS などの組み込み処理、autoprefixer、CSS minify。

意味: `.css.njk` は Nunjucks で文字列を生成するだけで、CSS compiler ではありません。平文 `.css` は byte copy です。

必要な場合: preprocessor が必要なら、別工程で生成した CSS を JSKim の source に置く方法を検討します。

## 独自の plugin システムはない

提供しないもの: JSKim 専用の plugin / extension 機構。

意味: 拡張の主な手段は `jskim.config.js` の `nunjucks.filters` / `nunjucks.globals` です。これは Nunjucks への登録であり、汎用 plugin marketplace ではありません。

## Build は依存グラフに基づく増分 build ではない

提供しないもの: file 単位の依存解析による incremental build。

意味: 変更検知後は、設定に従い clean（既定）付きの全体再 build が基本です。大規模 project では、変更のたびに全体を作り直す前提で設計します。

速度の優劣を数値で断定はしません。

## Multi-project の command 制限

| command | --all |
|---------|-------|
| build   | 対応    |

| command | --all |
|---------|-------|
| watch   | 非対応   |
| serve   | 非対応   |
| dev     | 非対応   |

複数 project を同時に `watch` / `serve` / `dev` したい場合は、別 process で個別に起動する必要があります。

## Port の扱い

- 既定は `127.0.0.1:3000`
- 衝突時に次の空き port を自動選択しない
- `--port` または `serve.port` で変更する
- process manager 機能はない

詳細は [エラーとトラブルシューティング](#) を参照してください。

## Browser open

- `dev --open` のみ
- `serve --open` は非対応
- OS の既定ブラウザ起動に依存
- 失敗時は warning で、開発サーバーは継続

## CSS soft reload の範囲

対象になり得る条件（概要）：

- `.css` / `.css.njk` の `change`
- `templates[]` 配下ではない
- same-origin の `<link rel="stylesheet">`

対象外・制限の例:

- external stylesheet
- `<style>` inline
- 動的挿入 stylesheet（保証なし）
- soft reload 失敗時は full reload

詳細は [開発機能](#) を参照してください。

## Live reload は HMR ではない

- page の full reload、または CSS の差し替え
- JavaScript module の実行状態保持は保証しない
- React / Vue などの HMR と同種の機能ではない

## Browser overlay と環境依存

- `dev` + live reload 時に HTML へ script を注入
- overlay は Shadow DOM を使用

Strict な Content-Security-Policy など、ブラウザや page のセキュリティ設定によっては、注入や接続が制限される可能性があります。

すべての CSP 設定での動作を保証するものでも、必ず失敗すると断定するものでもありません。

## Config と Nunjucks の境界

- 余分な config key は無視されることがある（厳密 schema ではない）
- `autoescape` / `noCache` は JSKim が固定
- `data.rootPath` は予約
- `dev` では一部設定変更に process 再起動が必要

Nunjucks 全体の機能を JSKim の契約として保証しません。

公式ガイドは sample で有用な文法が中心です（[Nunjucksの使い方](#)）。

## Output directory

- `dist` は source ではない
- `build.clean` が `true`（既定）なら build 前に削除される
- `false` の場合、消えた source に対応する古い output が残ることがある
- 手編集の維持は保証しない

## Platform / path

- Windows では大文字小文字違いの output 衝突を検出する
- path は Node の `path` で扱い、HTML 内は `/` を使う
- Windows / macOS / Linux を特別に排除していない
- `--open` の挙動は OS の handler に依存する

## 公式 sample の範囲

- 学習用の静的 UI sample

- production-ready な application template ではない
- デザインシステム / UI ライブラリではない
- セキュリティや業務要件を満たすことを保証しない

## ツール選択の目安

| 必要                       | 判断                              |
|--------------------------|---------------------------------|
| Nunjucks で静的 page を生成したい | JSKim が適合しやすい                   |
| API / database server    | 別途 backend が必要                  |
| JS bundling / transpile  | 別途 bundler が必要                  |
| 既存 HTML の自動 migration    | 別作業 / 別ツールが必要                   |
| 本格的な SPA                 | 目的に合う application framework を検討 |

特定製品の推奨や比較評価は、この章の範囲外です。

関連する運用上の確認は [エラーとトラブルシュート](#)、画面例は [Dashboard例](#) 以降を参照してください。